

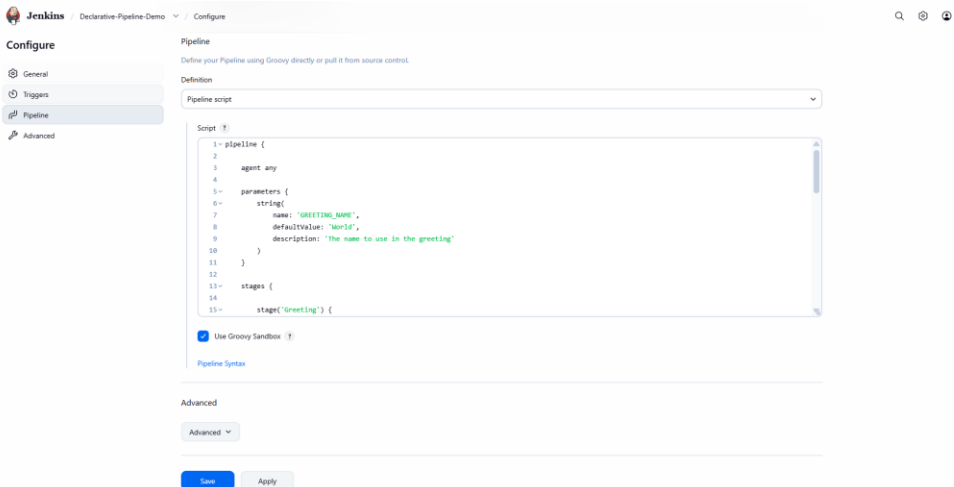


SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

Department of Computer Science and Engineering DevOps Lab - L3

Lab Assignment No: 7	
Name of Student	Sanyukt Ramola
PRN	23070122194
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Title of Assignment	Pipeline with Parameters
Theory	Declarative Pipeline with Parameters A Declarative Pipeline is a Jenkins feature that allows the build, test, and deployment process to be defined as code using a structured Groovy syntax. It provides a simple and readable way to automate software development workflows. Pipeline parameters enable users to provide input values at the time of execution, making the pipeline flexible and reusable. In this experiment, a string parameter (GREETING_NAME) was added to the pipeline, allowing the user to enter a custom value during execution. The pipeline accessed this value using the params object and displayed it in the console output. This demonstrates how parameterized pipelines can accept user input and perform dynamic operations based on the provided values.
Implementation	Step 1: Open the Existing Pipeline Project  <pre>1 pipeline { 2 agent any 3 parameters { 4 string(5 name: 'GREETING_NAME', 6 defaultValue: 'World', 7 description: 'The name to use in the greeting' 8) 9 } 10 stages { 11 stage('greeting') { 12 // ... 13 } 14 } 15 }</pre>

Opened Pipeline Configuration — Opened the configuration page of the existing Declarative-Pipeline-Demo project to modify the pipeline script and add build parameters.

Step 2: Add the Parameters Block & Save



Modified Script:

```
pipeline {
    agent any

    parameters {
        string(
            name: 'GREETING_NAME',
            defaultValue: 'World',
            description: 'The name to use in the greeting'
        )
    }

    stages {
        stage('Checkout Source Code') {
            steps {
                echo 'Starting Git checkout...'
                git url: 'https://github.com/riavinod10/devops-lab', branch: 'main'
                echo 'Git checkout completed.'
            }
        }

        stage('Run Example Command') {
            steps {
                echo "Hello ${params.GREETING_NAME}"

                sh 'echo "Hello from Declarative Pipeline!"'
                sh 'echo "--- Reading README ---"'
                sh 'cat README.md'
            }
        }
    }

    post {
        always {
            echo 'Pipeline finished.'
        }
    }
}
```

Configured Parameterized Pipeline — Added the GREETING_NAME string parameter to the Declarative Pipeline, updated the pipeline script to use the parameter in the greeting message, and saved the updated pipeline configuration.

Step 3: Build with Parameters:

Jenkins / Declarative-Pipeline-Demo

Pipeline Declarative-Pipeline-Demo

This build requires parameters:

GREETING_NAME
The name to use in the greeting

World

Build Cancel

Builds

Filter

Today

- #2 12:18 PM
- #1 10:39 AM

Provided Build Parameter — Used the Build with Parameters option and entered the value for the GREETING_NAME parameter before executing the Declarative Pipeline.

Step 4: Execute the Parameterized Pipeline:

Jenkins / Declarative-Pipeline-Demo

Declarative-Pipeline-Demo

Permalinks

- Last build (#2), 3 min 27 sec ago
- Last stable build (#2), 3 min 27 sec ago
- Last successful build (#2), 3 min 27 sec ago
- Last completed build (#2), 3 min 27 sec ago

Builds

Filter

Today

- #2 12:18 PM
- #1 10:39 AM

Executed Parameterized Pipeline — Successfully executed the parameterized Declarative Pipeline using the provided GREETING_NAME value and verified that the build completed successfully.

Step 5: Verify the Parameter Usage:

```
[Pipeline] echo
Git checkout completed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Example Command)
[Pipeline] echo
Hello World
[Pipeline] sh
+ echo Hello from Declarative Pipeline!
Hello from Declarative Pipeline!
[Pipeline] sh
+ echo --- Reading README ---
--- Reading README ---
[Pipeline] sh
+ cat README.md
🔒🔒 My Awesome DevOps Lab (Resolved Conflict)

This update adds details about a new feature.

[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finished.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Verified Parameter Usage — Examined the console output and verified that the pipeline successfully used the GREETING_NAME parameter value (World) during execution. The pipeline completed successfully with a Finished: SUCCESS status.

Subject Faculty Name: Prof. Arijit Dutta